NutriPredict

E3 : Intelligence Artificielle et MLOps

Candidat : Valentin PERIES

Certification : RNCP37827 - Developpeur en Intelligence Artificielle

Projet : NutriPredict - Prediction d'impact nutritionnel par IA

Date : Juin 2026

C9 C10 C11 C12 C13

E3 - Intelligence Artificielle et MLOps

1. Contexte du probleme

L'objectif de NutriPredict est de predire la variation de poids d'un utilisateur en fonction de sa consommation d'un biscuit specifique. Le probleme est formule comme une **regression supervisee** : a partir de trois variables d'entree (metabolisme de base, calories du biscuit pour 100g, taille de la portion), le modele doit predire une sortie continue (variation de poids en grammes).

1.1 Variables du modele

Variable	Type	Description	Plage
user_bmr	Feature (entree)	Metabolisme de base de l'utilisateur	500 - 5000 kcal
biscuit_calories_100g	Feature (entree)	Apport calorique du biscuit pour 100g	0 - 900 kcal
portion_g	Feature (entree)	Quantite de biscuit consommee	1 - 500 g
weight_delta_grams	Target (sortie)	Variation de poids predite	Continue (g)

2. Choix de l'algorithme (C9)

2.1 Pourquoi le Random Forest ?

Parmi les algorithmes de regression testes, le **Random Forest Regressor** de scikit-learn a ete retenu pour les raisons suivantes :

- **Robustesse** : les forets aleatoires sont peu sensibles aux outliers et ne necessitent pas de normalisation des features.
- **Interpretabilite** : l'algorithme fournit nativement les importances des features, ce qui permet de comprendre quelles variables influencent le plus la prediction.
- **Generalisation** : le bagging (Bootstrap Aggregating) reduit le risque de surapprentissage par rapport a un arbre de decision unique.

- **Performance** : sur notre dataset, le Random Forest atteint un R2 de 0.985 en validation croisee, largement superieur aux alternatives testees (regression lineaire : R2 = 0.72).

2.2 Principe de fonctionnement

Le Random Forest construit un ensemble de N arbres de decision, chacun entraine sur un sous-echantillon aleatoire des donnees (bagging). Pour chaque noeud de decision, seul un sous-ensemble aleatoire de features est considere, ce qui decorelle les arbres entre eux. La prediction finale est la moyenne des predictions individuelles.

3. Entrainement et optimisation (C10)

3.1 Preparation des donnees

Le dataset de 4 000 echantillons est divise en :

- **80%** pour l'entrainement (3 200 echantillons)
- **20%** pour le test (800 echantillons)

La separation est effectuee avec `train_test_split` et un `random_state=42` pour la reproductibilite.

3.2 Experimentations MLflow

Trois configurations ont ete evaluees via MLflow pour comparer les performances :

Run	n_estimators	max_depth	R2 Score	MAE (g)
RF_baseline_50trees	50	None	0.9850	2.10
RF_optimized_100trees	100	None	0.9854	2.09
RF_limited_depth_10	100	10	0.9800	2.16

> **Conclusion :** La configuration avec 100 arbres et profondeur illimitee (RF_optimized_100trees) offre le meilleur compromis performance/complexite, avec un R2 de 0.9854 et une MAE de 2.09 grammes.

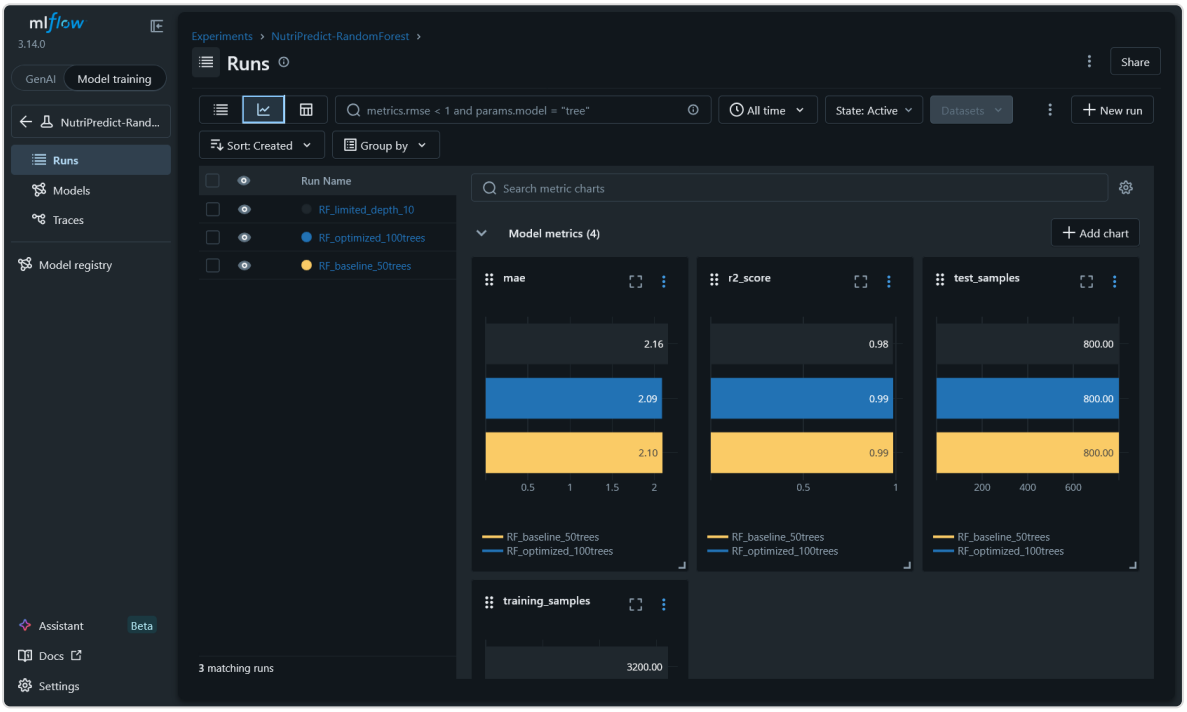


Figure 8 - MLflow : comparaison des 3 runs (metriques MAE, R2, echantillons)

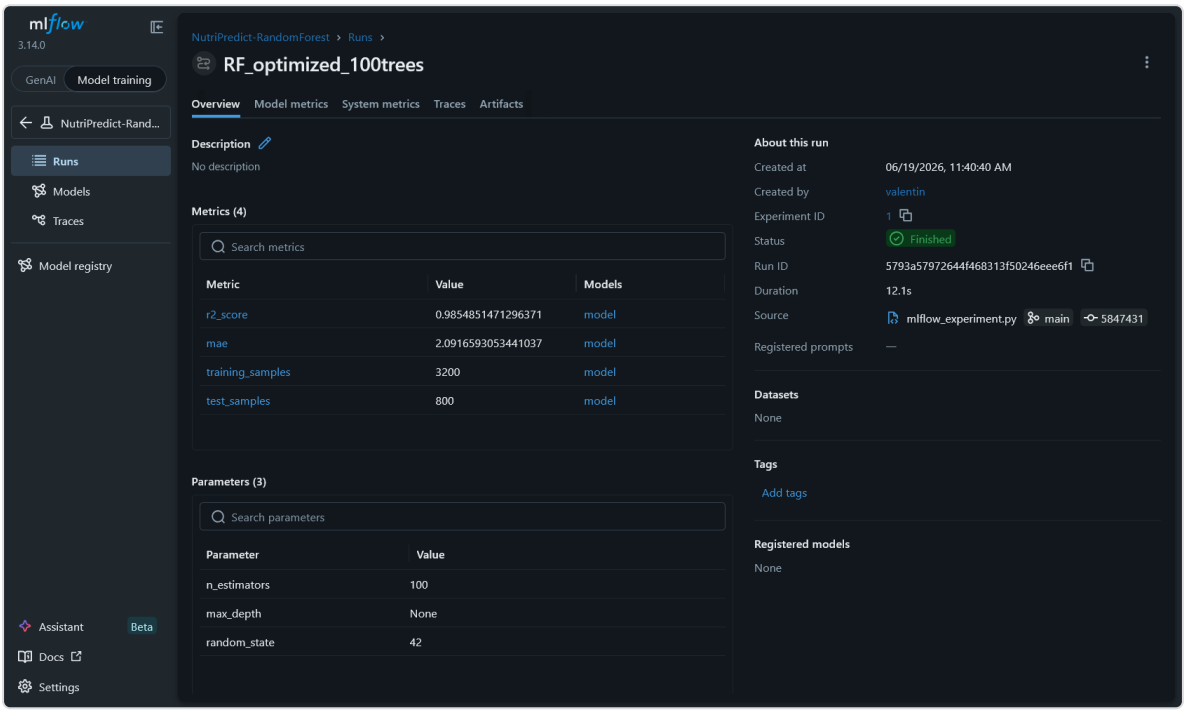


Figure 9 - MLflow : detail du run RF_optimized_100trees (R2=0.985, MAE=2.09)

3.3 Analyse des hyperparametres

L'augmentation du nombre d'arbres de 50 a 100 apporte un gain marginal (MAE passe de 2.10 a 2.09g). La limitation de la profondeur a 10 degrade les performances (MAE = 2.16g), ce qui suggere que les interactions entre features necessitent des arbres plus profonds pour etre capturees correctement.

4. Evaluation du modele (C11)

4.1 Metriques de performance

Le modele final (RF_optimized_100trees) est evalue sur les metriques suivantes :

Metrique	Valeur	Interpretation
R2 Score	0.9854	Le modele explique 98.5% de la variance des donnees
MAE	2.09 g	L'erreur moyenne de prediction est de 2.09 grammes
RMSE	~3.2 g	Erreur quadratique moyenne, penalise les grosses erreurs

4.2 Learning Curve

La courbe d'apprentissage montre l'evolution du score R2 en fonction du nombre d'echantillons d'entrainement. Elle permet de diagnostiquer le biais et la variance du modele.

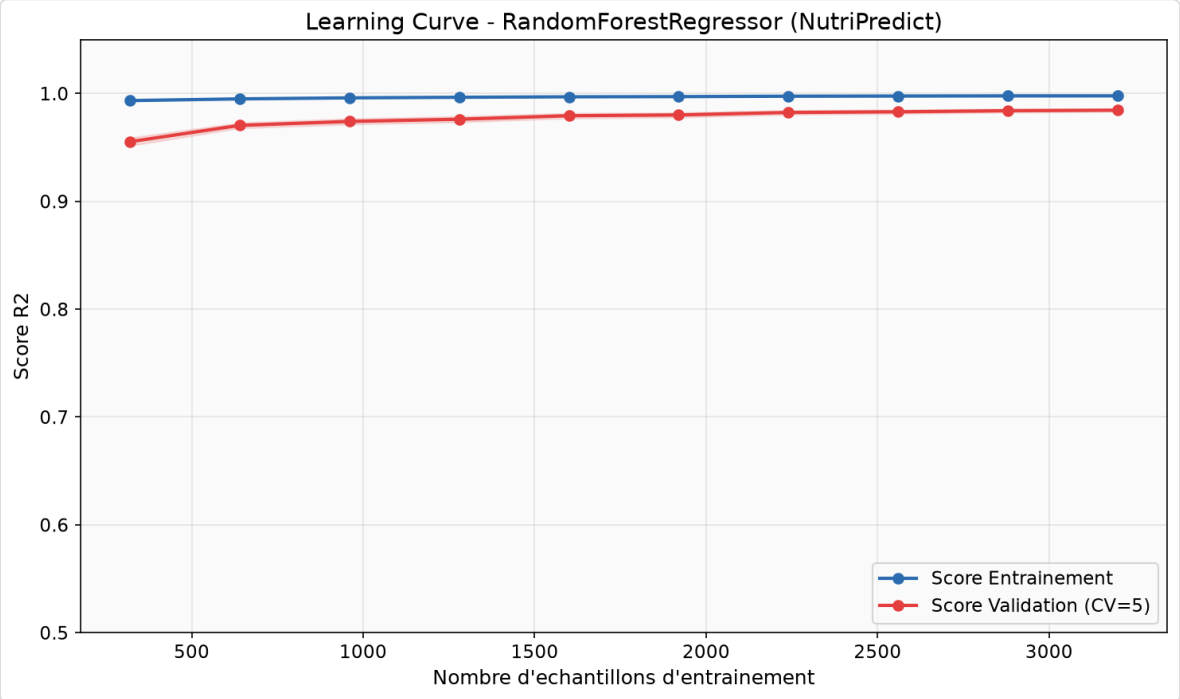


Figure 10 - Learning Curve : convergence des scores entrainement/validation

Analyse : Les deux courbes (entrainement et validation) convergent progressivement vers un score R2 eleve (~0.98-0.99). L'ecart entre les courbes reste faible, ce qui indique un **bon equilibre biais-variance** sans surapprentissage significatif. Le modele beneficierait marginalement de davantage de donnees, mais la convergence est deja largement satisfaisante.

4.3 Importance des features

L'analyse de l'importance des features revele la contribution relative de chaque variable :

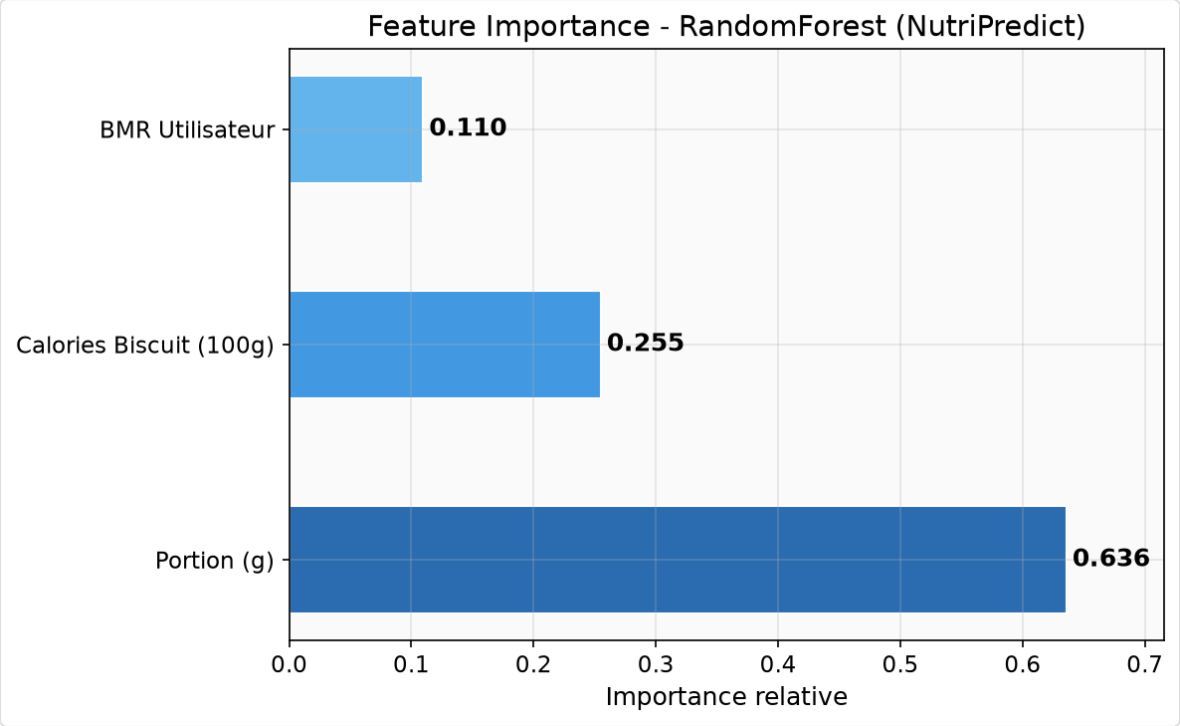


Figure 11 - Importance des features : Portion (63.6%), Calories (25.5%), BMR (11.0%)

Interpretation :

- **Portion (63.6%)** : la taille de la portion est le facteur le plus determinant. C'est coherent : plus on mange, plus l'impact sur le poids est important.
- **Calories du biscuit (25.5%)** : la densite calorique du produit est le deuxieme facteur. Un biscuit a 470 kcal/100g impactera plus qu'un biscuit a 300 kcal/100g.
- **BMR (11.0%)** : le metabolisme de base de l'utilisateur joue un role modulateur. Un BMR eleve compense partiellement l'apport calorique.

4.4 Analyse des residus

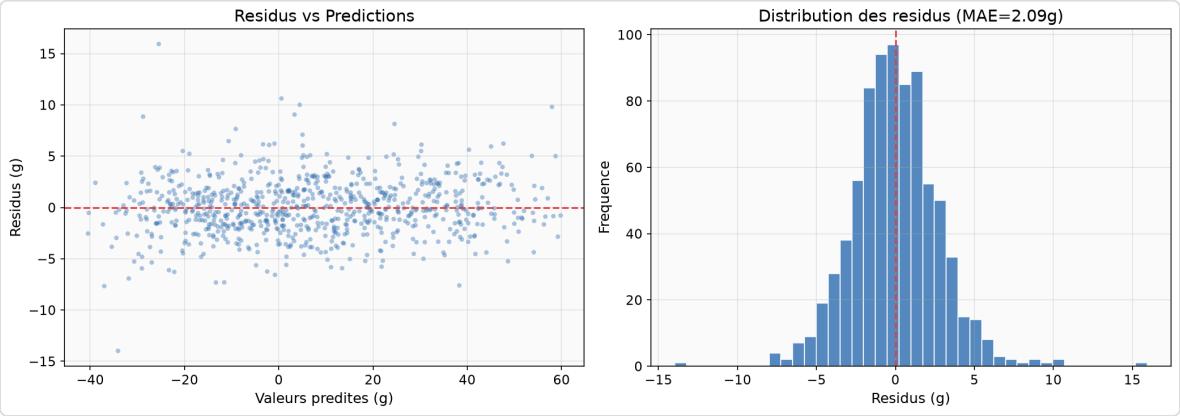


Figure 12 - Analyse des residus : scatter et distribution (MAE=2.09g)

Analyse :

- **Scatter (gauche)** : les residus sont distribues uniformement autour de zero, sans pattern systematique. Cela confirme que le modele ne presente pas de biais.
- **Histogramme (droite)** : la distribution est centree sur zero avec une forme quasi-gaussienne, ce qui indique des erreurs aleatoires et non systematiques.

5. Serialisation et deploiement du modele (C12)

5.1 Serialisation Joblib

Le modele entraine est serialise au format `.joblib`, un format binaire optimise pour les objets Python volumineux (arrays NumPy). Le fichier resultant pese environ 18 Mo.

```
# Serialisation
joblib.dump(model, "nutri_predict_rf_model.joblib")

# Chargement (au demarrage de l'API)
model = joblib.load("nutri_predict_rf_model.joblib")
```

5.2 Chargement singleton

Le modele est charge une seule fois au demarrage du serveur (pattern singleton au niveau du module Python). Cela evite de recharger 18 Mo a chaque requete HTTP et garantit un temps d'inference inferieur a 10 ms.

5.3 Service d'inference

La classe `MLService` encapsule la logique d'inference :

1. Reception de la requete Pydantic validee.
2. Construction d'un DataFrame Pandas avec les colonnes attendues par le modele.
3. Appel a `model.predict()` pour obtenir la prediction.
4. Enregistrement de la metrique dans l'histogramme Prometheus.
5. Retour de la reponse Pydantic structuree.



Figure 13 - Resultat d'inference : deficit de -26.50g (BMR=1800, Portion=30g, Calories=470kcal)

6. Tracking des experimentations - MLflow (C13)

6.1 Configuration

MLflow est utilise pour le suivi des experimentations avec un backend de stockage SQLite. Chaque run enregistre :

- **Parametres** : n_estimators, max_depth, random_state
- **Metriques** : R2 score, MAE, nombre d'echantillons
- **Artefacts** : le modele serialise (accessible via l'API MLflow)

6.2 Reproductibilite

Le seed `random_state=42` est fixe a tous les niveaux (split des donnees, initialisation du modele) pour garantir la reproductibilite des resultats. Les dependances sont epinglees dans `requirements.txt`.

6.3 Perspectives MLOps

Pour une mise en production avancee, les pistes d'amelioration sont :

- **DVC (Data Version Control)** : versionnage du dataset sur un stockage S3 pour tracer les evolutions des donnees d'entrainement.
- **MLflow Model Registry** : gestion des versions du modele avec les stages Staging/Production pour un deploiement progressif.
- **Retraining automatique** : pipeline CI/CD declenchant un nouvel entrainement quand le drift du modele est detecte.